Types of Operating System

OS goals –

1. Maximum CPU Utilization – What does it mean?

Ans:

Maximum CPU utilization means the CPU should stay busy as much as possible.
A good scheduling algorithm tries to **reduce CPU idle time**, so the processor is always working on some job.

Example:

If a scheduler switches to another ready process immediately when one process waits for I/O, the CPU never sits idle → utilization increases.

2. Less Process Starvation – What does it mean?

Ans:

Starvation happens when a process keeps waiting for CPU for a long time because other processes are always chosen before it.

A good scheduling algorithm ensures **every process eventually gets CPU time**, especially low-priority ones.

Example:

Aging technique increases priority of old waiting processes to prevent starvation.

3. Higher Priority Job Execution – What does it mean?

Ans:

In priority-based scheduling, tasks with higher priority should be executed first.
This ensures important, time-critical, or urgent processes get the CPU earlier than normal processes.

Example:

A system process like handling keyboard interrupts runs before a background music app because it has higher priority.

1.Single Process Operating System

A **Single Process Operating System** is a system where the OS can run **only one process at a time**.

At any moment, only **one program** is loaded into the memory and only that program gets the CPU's attention.

This type of OS does **not support multitasking**. As soon as one program finishes, the next one can start.

How it Works (Simple Explanation)

- The OS loads one program into memory.
- The CPU executes it from start to end.
- Until the program finishes, no other program can run.
- After completion, the next program is loaded.

There is **no concept of multiple processes running together**, no process switching, and no background tasks.

Examples

- **MS-DOS** (Microsoft Disk Operating System)
- **CP/M (Control Program for Microcomputers)**

Advantages of Single Process OS

1. Simple to Design and Use

The structure is easy to understand. With only one process, the OS design remains straightforward.

2. Very Low Overhead

No need for process scheduling, context switching, or CPU sharing.
This reduces memory usage and speeds up execution.

3. Predictable Behavior

Since only one program runs, its execution time is easier to estimate.
Ideal for simple, dedicated tasks.

4. Fast Startup

Such systems boot quickly because they load fewer components and have no complex scheduling.

Disadvantages of Single Process OS

1. No Multitasking

Users cannot run multiple programs at once.
For example: You cannot play music and type a document simultaneously.

2. Poor CPU Utilization

If the program waits for input/output, the CPU also sits idle.
This wastes processing power.

3. Not Suitable for Modern Workloads

Modern applications require multitasking, networking, background services, etc., which single process OS cannot handle.

4. Less Productivity

Having to close one program to open another slows down workflow.

5. No Process Protection

If the running program crashes, the whole system may crash.

Simple Real-Life Example

Imagine a shop with **only one employee**.
He handles one customer completely before calling the next.
If someone asks a quick question, they still have to wait until the first customer finishes.

This is exactly how a **Single Process OS** works.

2.Batch-Processing Operating System

A **Batch-Processing Operating System** is an OS where similar jobs are collected together, grouped into batches, and executed one after another **without user interaction**.
The user submits the job, and the system processes it later when its turn comes.

In this system, users do **not interact with the computer during execution**. Instead, the OS manages everything: job scheduling, loading, execution, and output generation.

How it Works (Simple Explanation)

1. Users submit jobs (program + data) to the computer.
2. The OS groups similar jobs into **batches**.
3. These batches run sequentially.
4. The OS decides the order of execution using job schedulers.
5. After completion, the output is returned to the user.

There is **no immediate execution**, and no real-time interaction.

Examples

- **IBM 1401** (classic example of batch processing)
- **Early mainframe systems**
- **Payroll processing systems**
- **Bank cheque processing**

Even today, many large organizations use batch processing for monthly or weekly tasks.

Advantages of Batch-Processing OS

1. Efficient for Large Jobs

Ideal for tasks that require heavy computation or run at scheduled intervals—like monthly billing or payroll.

2. Reduces CPU Idle Time

Jobs are arranged so the CPU always has something to execute.

3. No Need for User Interaction

Users can submit the job and leave. The system handles the rest.

4. Good for Repetitive Work

Tasks that repeat every day/week/month benefit from batching.

5. High Throughput

Because similar jobs are grouped together, the system completes more jobs in less time.

Disadvantages of Batch-Processing OS

1. No Real-Time Result

Users cannot get immediate output. They must wait until the batch is processed.

2. Hard to Debug

If a job has an error, you might not find out until the end of the batch.

3. Requires Experienced Operators

Batch systems need people who understand how to schedule and manage jobs.

4. Long Turnaround Time

Sometimes jobs take hours or days before they are executed.

5. Not Suitable for Interactive Tasks

Jobs like editing a document or browsing the web cannot be done in batch mode.

Simple Real-Life Example

Imagine a **post office** sorting letters.
They collect all letters throughout the day, **group them by city**, and then process each group in order.

Users do not stand there while the sorting happens.
They simply drop the letter and leave.

This is how a **Batch-Processing OS** works.

3.Multiprogramming Operating System

A **Multiprogramming Operating System** allows **multiple programs** to stay in memory at the same time, and the CPU switches between them to increase efficiency.
The main purpose is to **keep the CPU busy** as much as possible by executing another program when one program is waiting for I/O.

This system does **not** mean multiple programs run at the exact same moment, but they share the CPU efficiently.

How it Works (Simple Explanation)

1. Several programs are loaded into the main memory.
2. The CPU picks one program and starts executing it.
3. When that program waits for input/output (like reading a file),
   the CPU **switches to the next ready program**.
4. This continues, ensuring the CPU rarely sits idle.
5. The OS uses **job scheduling** and **memory management** to handle this switching.

So even though only one program runs on the CPU at a time, the switching happens so smoothly that it feels like multiple programs are running together.

Examples

- **UNIX**
- **Linux**
- **Modern versions of Windows**

Any system that allows running multiple applications (browser + music player + text editor) supports multiprogramming.

Advantages of Multiprogramming

1. Maximum CPU Utilization

When one job waits for I/O, another job uses the CPU.
This reduces idle time and improves system performance.

2. Better System Throughput

More jobs are completed in less time because the CPU stays busy.

3. Efficient Resource Usage

Memory, CPU, and devices are used fully because multiple jobs are active.

4. Multiprogramming Increases Productivity

Users can run multiple programs at once without waiting.

Disadvantages of Multiprogramming

1. Complex OS Design

Managing multiple processes together requires advanced scheduling and memory management.

2. May Cause Starvation

Low-priority jobs might wait too long if higher-priority jobs keep arriving.

3. Requires Large Memory

To hold multiple programs at once, more RAM is needed.

4. Security Issues

Since multiple programs share memory and resources, there is a higher chance of data leakage or misuse.

5. Difficult to Troubleshoot

Errors in one process can affect others if not managed properly.

Example: Using a Computer While Running Multiple Apps

Suppose you are doing these tasks on your laptop:

- A **YouTube** video is playing
- A **file is getting downloaded** in the browser

- You are **typing notes** in a text editor
- A **Python program** is running in the background

All these programs are inside memory at the same time.

When:

- The YouTube video waits for buffering (I/O wait) →
  CPU immediately switches to your text editor.
- While your Python program waits for input/output →
  CPU uses that time to continue video playback.
- When the download is waiting for network response →
  CPU switches to some other process.

The CPU keeps switching between all these programs so fast that everything seems to work smoothly together.

This is **multiprogramming in action**.

4.Multitasking Operating System

A **Multitasking Operating System** allows a user to perform **multiple tasks at the same time**, giving the feel that all programs are running together.
It works by quickly switching the CPU between different tasks so fast that the user experiences smooth, parallel activity.

In multitasking, the OS focuses on **user interaction** and **real-time responsiveness**.

How Multitasking Works (Simple Explanation)

1. Multiple programs are loaded into memory.
2. The CPU gives each program a small time slice (a few milliseconds).
3. After its time slice ends, the CPU switches to the next program.
4. Because this switching happens very fast, everything seems to run side by side.
5. Users can interact with all programs simultaneously.

This quick switching is called **context switching**.

Examples of Multitasking OS

- **Windows 10 / 11**
- **Linux (Ubuntu, Fedora, etc.)**
- **macOS**
- **Android**
- **iOS**

Any modern operating system that lets you open multiple apps at once supports multitasking.

Real-Life Example (Easy to Understand)

Example: Using a Smartphone

On your mobile:

- Watching YouTube
- Replying to WhatsApp messages
- Downloading files in the browser
- Music playing in the background

You are interacting with multiple apps at the same time.
The OS rapidly switches the CPU between these apps, so everything appears to work together.

This is **multitasking**.

Advantages of Multitasking

1. High User Productivity

You can perform many activities at once—editing documents, browsing, playing music, etc.

2. Fast and Smooth User Experience

The OS quickly switches between tasks, giving a seamless feel.

3. Better Resource Utilization

CPU, memory, and devices are used efficiently.

4. Suitable for Modern Applications

Most apps today require background processing + user interaction.

Disadvantages of Multitasking

1. Higher CPU Usage

Constant switching requires processing power.

2. More Memory Required

Multiple apps must stay loaded.

3. Context Switching Overhead

Frequent switching may slow down the system slightly.

4. Security Risks

Multiple programs running together increases the chance of vulnerabilities.

Simple Analogy

Imagine a person **doing homework**, **talking on the phone**, and **listening to music**.
They switch attention quickly among tasks, giving the feel of doing all at once.

This is how **multitasking OS** behaves.

5. What is Multiprocessing?

Multiprocessing means **multiple processors/CPUs working together at the same time** to execute different processes.

In old computers, everything ran on a **single CPU**, so tasks were done **one-by-one**.
Modern systems use **multiple CPUs/cores**, so several tasks can run **parallel**.

Example:
Watching YouTube + Running VS Code + Background virus scan.
Each process can run on a different CPU core.

How Multiprocessing Works

- OS creates **multiple processes**.
- These processes can run **truly parallel**, because each process gets its own CPU core.
- Each process has **separate memory**, so they don't interfere with each other.

Advantages of Multiprocessing

1. True Parallelism

Because multiple CPUs work together, tasks finish faster.

Example:
One core handles video processing, another handles downloading, another handles audio.

2. Higher System Throughput

Throughput = amount of work done in a given time.
More processors → more work completed.

Example:
Servers handling thousands of requests use multiprocessing to keep everything smooth.

3. More Responsive System

Even heavy tasks won't freeze your system, because other CPUs are free for other tasks.

Example:
Rendering a video won't stop you from browsing the internet.

4. Better Reliability

Some multiprocessing systems use **redundant CPUs**.
If one CPU fails, work shifts to another.

Example:
Banking servers run multiple CPUs so that failure does not stop transactions.

5. Suitable for CPU-Heavy Tasks

Great for tasks that need high computation:

- AI/ML model training
- Video rendering
- Scientific simulations
- Large mathematical calculations

Disadvantages of Multiprocessing

1. Expensive Hardware

Multiple processors increase the system cost.

Traditional example:
Mainframe systems used in banks — very costly compared to single-CPU systems.

2. Complex OS and Process Management

OS must handle:

- Process creation
- Scheduling on different CPUs
- Inter-process communication

- Deadlock handling

This makes OS design more complicated.

3. Synchronization Problems

Processes often need to share data.
To avoid conflicts, OS must use:

- Locks
- Semaphores
- Monitors

These slow down system performance if not managed properly.

Example:
Two processes writing to the same bank account record → incorrect balance.

4. Memory Management Becomes Difficult

Each process needs its own memory.
Managing multiple memory spaces is complex and may cause:

- Fragmentation
- High memory usage
- More cache misses

5. Increased Power Consumption & Heat

More CPUs → more electricity used → more heat generated.

Example:
High-end servers need special cooling systems.

Advantages:

- True parallelism
- Faster processing
- Better responsiveness
- High reliability
- Best for CPU-heavy tasks

Disadvantages:

- Expensive hardware
- Complex OS design
- Synchronization issues
- Complicated memory management

- Higher power usage
- Some tasks can't be parallelized
- Deadlocks are more common

6.What is a Distributed Operating System?

A **Distributed OS** is an operating system that runs on **multiple computers** but works as if it is **one single system**.

Multiple machines = multiple CPUs, memories, and storage
But the OS coordinates them so smoothly that the user feels everything is one big computer.

Understanding

Imagine you have **4 computers** connected.
Distributed OS makes them behave like **one powerful computer**.

Examples

- **Google Search servers**
- **Amazon AWS cloud**
- **Microsoft Azure**
- **Hadoop cluster**
- **Kubernetes clusters**
- **Facebook data centers**

How It Works

Distributed OS coordinates:

- File sharing
- Process execution
- Communication
- Load balancing
- Fault tolerance

among all connected machines.

Advantages of Distributed OS

1. High Performance

Work is divided among multiple computers → tasks finish faster.

Example:
Google splits your search query across thousands of servers and gets results in milliseconds.

2. High Reliability

If one machine fails, others continue.

Example:
In Amazon servers, if one node dies, the work shifts automatically.

3. Resource Sharing

Computers share:

- Files
- Printers
- Storage
- Network

Example:
All nodes in a cluster can access the same file system.

4. Scalability

You can add more machines to increase power.

Example:
If a website gets more users, you add more servers to handle the load.

5. Parallel Processing

Distributed systems allow real parallelism — many machines perform different parts of the same task.

Example:
Hadoop splits a big data file into chunks and processes them in parallel.

Disadvantages of Distributed OS

1. Complex Design

Building a distributed OS is very difficult.

You must handle:

- Communication between machines

- Shared memory issues
- Clock synchronization

2. Network Dependency

If the network fails → entire system may stop.

Example:
If network switch fails in a data center, all servers can't communicate.

3. Security Issues

More computers → more risks.

- Unauthorized access
- Data leakage
- Network attacks

4. Expensive Setup

Distributed systems need:

- High-speed networks
- Many servers
- Skilled engineers

This increases cost.

5. Debugging is Difficult

Finding errors across multiple machines is very tough.

Example:
A small bug on one node can affect the whole cluster.

RTOS Definition

RTOS (Real-Time Operating System) is a type of operating system that runs tasks **on time** and gives **quick and predictable responses**.
It makes sure every task finishes **within a fixed time** (deadline).

It is used in systems where **even a small delay can cause failure**, like robots, medical devices, cars, machines, etc.

Advantages

- **Fast response** to events.
- **Works on fixed timing** (predictable).
- **High reliability** for critical systems.
- **Efficient CPU usage.**
- **Lightweight** and small in size.

Disadvantages

- **Difficult to design** and program.
- **Limited features** compared to normal OS.
- **Not good for too many tasks.**
- **Needs expert developers.**
- **Timing bugs are hard to fix.**

Feature	**Multiprogramming**	**Multitasking**	**Multiprocessing**
Meaning	Multiple programs are kept in memory and the CPU switches between them.	A single user can run many tasks at the same time.	System has **multiple CPUs** that work together.
Goal	Maximize CPU usage.	Allow user to do many things at once.	Increase speed and handle heavy work.
Type of system	Old batch systems, early OS.	Modern desktop/laptop OS.	High-performance and server systems.
CPU	One CPU	One CPU	Two or more CPUs
Example	You are downloading a file and simultaneously a compiler is running in background (CPU switches between them).	You listen to music while typing a document and browsing.	Intel i5/i7 with multiple cores, servers running many CPUs together.
User involvement	Less	High	Medium
Performance	Medium	High for user tasks	Very high for parallel tasks